COLLEGE PROJECT DOCUMENTATION

# RAG Assistant: A Retrieval-Augmented Generation Document Assistant

## Project Plan using Agile Methodology

| | |
|---|---|
| Document | Project Plan using Agile Methodology |
| Version | 1.0 |
| Date prepared | 12 October 2026 |
| Prepared by | |
| Course / Supervisor | |

Planning document. The project is estimated and planned from Step 0: no development is assumed to be complete.

# Contents

1. Introduction
2. Agile Methodology
3. Product Backlog
4. Sprint Plan
5. Sprint Timeline
6. Agile Roles and Responsibilities
7. Testing Strategy
8. Risk Management
9. Agile Metrics and Monitoring
10. Final Deliverables and Acceptance Criteria
11. Conclusion

> This plan describes planned work starting at Step 0. No sprint has been run yet, so it contains no velocity, completion or test-result data. It covers the same scope and technology stack as the companion document *Project Estimation using the COCOMO Model*.

# 1. Introduction

## 1.1 Project overview

The RAG Assistant is a web application where users upload documents and ask questions about them. The system retrieves relevant passages with dense vector search and BM25 keyword search, combines and reranks them, and uses an LLM to write an answer with source citations. It is built with Next.js, TypeScript and Tailwind CSS (frontend), Python and FastAPI (backend), PostgreSQL, Qdrant and the BAAI/bge-small-en-v1.5 embedding model.

## 1.2 Project objectives

1. Deliver a working increment at the end of every sprint.
2. Build the core RAG path (upload, index, retrieve, answer, cite) before enhancements.
3. Secure user data with authentication and isolation.
4. Test continuously so that each increment is reliable.
5. Finish with a documented, demonstrable application.

## 1.3 Intended users

- **Students** working with lecture notes, papers and textbooks.
- **Researchers and professionals** who need verifiable answers from reports and manuals.
- **Evaluators** (supervisor, examiners) who review the application and documentation.

## 1.4 Expected deliverables

A working application, source code, automated tests and test evidence, UML diagrams, README, project report with screenshots, and a demonstration (Section 10).

# 2. Agile Methodology

The project follows an **Agile, Scrum-inspired** approach adapted for one student. Work is delivered in short, fixed-length sprints, each producing a tested, working increment, instead of completing all design, then all coding, then all testing.

| Practice | How it is applied in this project |
|---|---|
| Iterative and incremental development | Four two-week sprints. Each sprint adds a usable slice: ingestion, then question answering, then the web interface, then quality improvements. |
| Product backlog | A single prioritised list of user stories and technical tasks (Section 3), ordered by MoSCoW priority and dependencies, and refined at every sprint planning. |
| Sprint planning | At the start of each sprint the student selects backlog items that fit the sprint's capacity (about 30 hours), states a sprint goal and breaks items into tasks. |
| Daily progress tracking | A solo replacement for the daily stand-up: a short written log each working day (done yesterday, plan today, blockers) and a task board (To do / In progress / Done). |
| Sprint review | At the end of each sprint the increment is demonstrated against the acceptance criteria, ideally to the supervisor, and feedback is added to the backlog. |
| Retrospective | A short written reflection after each review: what went well, what did not, and one or two concrete improvements for the next sprint. |
| Continuous testing and feedback | Tests are written with the code in every sprint and run before each commit. Review feedback and defects become backlog items. |

## 2.1 Why Agile suits a RAG application

- **Uncertain retrieval quality:** how well chunking, embeddings and ranking work is only known after testing with real documents. Short sprints let the student measure and adjust early.
- **Integration risk:** the system combines a web frontend, API, two databases, local models and an external LLM. Delivering a thin end-to-end path by Sprint 2 exposes integration problems early.
- **Estimate uncertainty:** the COCOMO estimate suggests more effort than the student schedule allows. Fixed time boxes with prioritised scope ensure the Must-have features are finished first, and lower-priority items are dropped rather than the deadline slipping.

# 3. Product Backlog

**Priority (MoSCoW):** Must-have = required for a passing submission; Should-have = important, planned, but can be postponed; Could-have = only if time remains.

**Story points (SP)** use the Fibonacci scale 1, 2, 3, 5, 8, 13. They express **relative complexity and uncertainty** compared with other items, not hours. A 5-point item is expected to be more complex than a 3-point item, but no fixed number of hours is attached to either.

| ID | User story / task | Priority | SP | Acceptance criteria | Depends on | Sprint |
|---|---|---|---|---|---|---|
| PB-01 | As the developer, I need the repository, Python/Node environments and Docker services (PostgreSQL, Qdrant) set up so that development can start. | Must | 3 | Backend health endpoint returns OK; both database containers start with one command; setup steps recorded. | - | S1 |
| PB-02 | As the developer, I need the architecture, API endpoints and database schema designed so that modules fit together. | Must | 3 | Schema creates users, documents, conversations and messages tables; endpoint list and Qdrant payload documented. | PB-01 | S1 |
| PB-03 | As a user, I want to register and log in so that my documents are private to me. | Must | 5 | Passwords stored only as salted hashes; valid login returns a token; wrong password returns 401; duplicate email returns 409. | PB-02 | S1 |
| PB-04 | As a user, I want every endpoint to check my identity so that nobody else can see my data. | Must | 5 | Requests without a valid token are rejected (401/403); a second user cannot list, query or delete the first user's data. | PB-03 | S1 |
| PB-05 | As a user, I want to upload PDF, TXT and Markdown files so that I can ask questions about them. | Must | 5 | Valid files are accepted and stored; other types, empty files and files over 50 MB are rejected with a clear message. | PB-02, PB-04 | S1 |
| PB-06 | As the system, I need to extract text from uploaded files so that it can be searched. | Must | 3 | PDF text is extracted page by page; TXT/MD read with encoding fallback; unreadable files produce an error, not a crash. | PB-05 | S1 |
| PB-07 | As the system, I need to split text into overlapping chunks so that retrieval returns focused passages. | Must | 2 | Chunks have size and overlap limits, keep page numbers and order; invalid chunk settings are rejected. | PB-06 | S1 |
| PB-08 | As the system, I need embeddings for each chunk (bge-small-en-v1.5) so that meaning-based search works. | Must | 3 | Each chunk gets a normalised 384-dimension vector; empty input is handled. | PB-07 | S2 |
| PB-09 | As the system, I need chunks indexed in Qdrant with user and document IDs so that search can be filtered. | Must | 5 | Uploaded document's chunks are stored and searchable; search never returns another user's chunks. | PB-08 | S2 |
| PB-10 | As a user, I want to list and delete my documents so that I can manage my library. | Must | 3 | List shows only my documents; delete removes metadata, vectors and the stored file. | PB-05, PB-09 | S2 |

| ID | User story / task | Priority | SP | Acceptance criteria | Depends on | Sprint |
|---|---|---|---|---|---|---|
| PB-11 | As a user, I want my question matched to relevant passages so that answers use the right content. | Must | 5 | Question returns the top-k chunks from my documents, optionally limited to one document. | PB-09 | S2 |
| PB-12 | As a user, I want an answer generated only from my documents so that it is trustworthy. | Must | 5 | Answer is based on retrieved context; when nothing relevant is found the system says so instead of guessing. | PB-11 | S2 |
| PB-13 | As a user, I want citations with each answer so that I can verify it. | Must | 3 | Every citation marker refers to a returned source with document, page and chunk; source text is included. | PB-12 | S2 |
| PB-23 | As the developer, I need an automated test setup with unit and API tests for ingestion and RAG so that changes are safe. | Must | 3 | pytest suite runs with one command; parsers, chunking, retrieval and query endpoint have tests; LLM is faked in tests. | PB-07, PB-12 | S2 |
| PB-14 | As a user, I want my questions and answers saved in conversations so that I can return to them. | Should | 5 | Messages are stored per conversation; I can list, reopen and delete my conversations; others cannot access them. | PB-04, PB-12 | S3 |
| PB-18 | As a user, I want login and registration pages so that I can use the app in a browser. | Must | 3 | Sign-in stores the session; expired or invalid sessions return to the login page with a message. | PB-03 | S3 |
| PB-19 | As a user, I want an upload screen and document library so that I can manage files visually. | Must | 5 | Drag-and-drop upload with progress; unsupported files rejected before upload; list and delete work. | PB-10, PB-18 | S3 |
| PB-20 | As a user, I want a chat screen that shows answers with clickable citations and sources. | Must | 8 | Question, loading and error states shown; citation markers link to source snippets; can scope a question to one document. | PB-13, PB-18 | S3 |
| PB-21 | As a user, I want a history sidebar so that I can switch between conversations. | Should | 3 | Sidebar lists my conversations, newest first; selecting one loads its messages; delete asks for confirmation. | PB-14, PB-20 | S3 |
| PB-15 | As a user, I want follow-up questions understood in context so that I can ask 'what about it?'. | Could | 2 | Follow-up questions are rewritten using recent history; on failure the original question is used. | PB-14 | S4 |
| PB-16 | As a user, I want keyword matches found too (BM25 + RRF hybrid retrieval) so that exact terms are not missed. | Should | 5 | Results combine dense and BM25 rankings with RRF; falls back to dense results if no keyword matches. | PB-11 | S4 |
| PB-17 | As a user, I want the best passages ranked first (cross-encoder reranking) so that answers improve. | Should | 3 | Candidates are reordered by the reranker; if the model fails, the previous order is kept. | PB-16 | S4 |
| PB-22 | (Non-functional) Errors are handled consistently so that users see helpful messages and data stays consistent. | Must | 3 | Invalid input gives 4xx with readable messages; service failures give an error without crashing; failed uploads leave no partial data. | PB-05, PB-12 | S4 |

| ID | User story / task | Priority | SP | Acceptance criteria | Depends on | Sprint |
|---|---|---|---|---|---|---|
| PB-24 | (Non-functional) Integration, user-isolation and regression tests so that the full system is verified. | Must | 3 | End-to-end checklist passes for upload, ask, cite, history; all automated tests pass before submission. | PB-20, PB-23 | S4 |
| PB-25 | (Non-functional) Uploads and processing stay responsive so that other requests are not blocked. | Should | 2 | Heavy parsing and embedding run off the request event loop; file size limit enforced. | PB-09 | S4 |
| PB-26 | As an evaluator, I want a README with setup instructions so that I can run the project. | Must | 2 | A new machine can install and run the app by following the README. | PB-24 | S4 |
| PB-27 | As an evaluator, I want UML diagrams, a report and screenshots so that I can assess the design. | Must | 3 | Class and activity diagrams match the code; screenshots cover login, upload, answer with citations, history. | PB-24 | S4 |
| PB-28 | As the student, I need a prepared demonstration so that the final presentation runs smoothly. | Must | 2 | Demo script rehearsed with sample documents and questions; fallback recording prepared. | PB-26, PB-27 | S4 |
| PB-29 | As an evaluator, I may want a public deployment so that I can try the app online. | Could | 5 | App reachable at a public URL with secrets kept out of the repository. | PB-28 | Unscheduled |

| Summary | Story points |
|---|---|
| Must-have items | 82 |
| Should-have items | 18 |
| Could-have items (planned in Sprint 4) | 2 |
| **Planned in Sprints 1-4** | **102** |
| Unscheduled Could-have (PB-29 deployment) | 5 |

## 3.1 Coverage of the COCOMO scope

Every module sized in the COCOMO estimate is covered by backlog items:

| COCOMO size module | Backlog items covering it | Sprint(s) |
|---|---|---|
| M1 Frontend web application | PB-18 to PB-21 | S3 |
| M2 Backend API layer | PB-01, PB-02, PB-22 | S1, S4 |
| M3 Document ingestion | PB-05 to PB-07 | S1 |
| M4 Embeddings and vector store | PB-08, PB-09 | S2 |
| M5 Retrieval and ranking | PB-11, PB-16, PB-17 | S2, S4 |
| M6 RAG generation and citations | PB-12, PB-13, PB-15 | S2, S4 |
| M7 Authentication, history and document management | PB-03, PB-04, PB-10, PB-14 | S1, S2, S3 |
| Non-functional work, testing, documentation | PB-22 to PB-28 | S2, S4 |

# 4. Sprint Plan

Four two-week sprints (8 weeks, 40 working days) at about 3 hours per working day give about 120 hours of capacity. Planned story points per sprint are 26, 27, 24 and 25 (total 102). Because there is no historical velocity yet, Sprint 1 also establishes a first velocity measurement, and later sprints are re-planned from it.

**Refinement of the suggested sequence:** authentication and user isolation were moved from Sprint 3 into Sprint 1, because uploads, documents and searches must belong to a user from the start; adding isolation later would mean reworking every endpoint. Document list and delete moved to Sprint 2 because they need the vector store.

## Sprint 1: Foundation and ingestion

| Aspect | Plan |
|---|---|
| Duration | 2 weeks, Weeks 1-2 (working days 1-10); about 30 hours of capacity |
| Sprint goal | A secured backend that accepts documents and turns them into page-tagged chunks. |
| Planned backlog items | PB-01 (3 SP); PB-02 (3 SP); PB-03 (5 SP); PB-04 (5 SP); PB-05 (5 SP); PB-06 (3 SP); PB-07 (2 SP) — total 26 story points |
| Development activities | • Clarify requirements; create the product backlog<br>• Set up repository, environments and Docker services<br>• Design architecture, API and database schema<br>• Implement registration, login, JWT and user isolation (moved forward from Sprint 3 because every upload must belong to a user)<br>• Implement upload validation, PDF/TXT/MD parsing and chunking |
| Testing and review | • Unit tests for parsers, chunking and password hashing<br>• API tests for auth and upload (valid, invalid type, empty, too large)<br>• Sprint review: upload a sample file and inspect its chunks |
| Sprint deliverable | Running backend with auth, upload, parsing and chunking |

## Sprint 2: Core RAG pipeline

| Aspect | Plan |
|---|---|
| Duration | 2 weeks, Weeks 3-4 (working days 11-20); about 30 hours of capacity |
| Sprint goal | Ask a question through the API and receive a grounded answer with citations. |
| Planned backlog items | PB-08 (3 SP); PB-09 (5 SP); PB-10 (3 SP); PB-11 (5 SP); PB-12 (5 SP); PB-13 (3 SP); PB-23 (3 SP) — total 27 story points |
| Development activities | • Embedding generation with bge-small-en-v1.5<br>• Qdrant indexing, filtered search, document list and delete<br>• Dense retrieval and LLM integration with a context-only prompt<br>• Context construction and citations<br>• Set up the automated test suite |
| Testing and review | • Unit tests for retrieval and citation building<br>• Initial API tests for query and documents (LLM faked)<br>• Manual check of answers on sample questions |
| Sprint deliverable | API that answers questions with citations (first end-to-end increment) |

## Sprint 3: Frontend and application features

| Aspect | Plan |
|---|---|
| Duration | 2 weeks, Weeks 5-6 (working days 21-30); about 30 hours of capacity |
| Sprint goal | A usable web application covering the full user journey. |
| Planned backlog items | PB-14 (5 SP); PB-18 (3 SP); PB-19 (5 SP); PB-20 (8 SP); PB-21 (3 SP) — total 24 story points |
| Development activities | • Next.js setup, login and registration pages<br>• Document upload interface and library<br>• Chat interface with citations and sources<br>• Conversation history (backend and sidebar)<br>• Frontend-backend integration |
| Testing and review | • Type checking and lint<br>• Frontend unit tests for the API client and citation helpers<br>• Manual browser validation of each user story |
| Sprint deliverable | Web app: sign in, upload, ask, view citations, reopen history |

### Sprint 4: Quality and submission

| Aspect | Plan |
|---|---|
| Duration | 2 weeks, Weeks 7-8 (working days 31-40); about 30 hours of capacity |
| Sprint goal | Improve retrieval, harden the system and prepare the submission. |
| Planned backlog items | PB-15 (2 SP); PB-16 (5 SP); PB-17 (3 SP); PB-22 (3 SP); PB-24 (3 SP); PB-25 (2 SP); PB-26 (2 SP); PB-27 (3 SP); PB-28 (2 SP) — total 25 story points |
| Development activities | • Hybrid retrieval (BM25 + RRF) and cross-encoder reranking<br>• Follow-up question rewriting (if time allows)<br>• Error handling and responsiveness improvements<br>• README, UML diagrams, report and screenshots<br>• Demonstration preparation |
| Testing and review | • Integration, user-isolation and regression tests<br>• Compare answers before and after reranking on the same sample questions<br>• Final acceptance review against all criteria |
| Sprint deliverable | Submission package and rehearsed demonstration |

## 4.1 Definition of Done (applies to every backlog item in every sprint)

1. Code for the backlog item is complete and committed to Git.
2. All acceptance criteria of the item are met and checked.
3. Unit tests and, where an endpoint is involved, API tests are written and passing.
4. No known Must-fix defects remain open for the item.
5. Errors are handled and user-facing messages are clear.
6. Relevant documentation (README notes, API notes) is updated.
7. The increment runs end to end with the work from earlier sprints.

# 5. Sprint Timeline

| Activity | Sprint 1 | | Sprint 2 | | Sprint 3 | | Sprint 4 | |
|---|---|---|---|---|---|---|---|---|
| | Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 | Week 8 |
| **Scrum events** | | | | | | | | |
| Sprint planning (start of each sprint) | | | | | | | | |
| Sprint review and retrospective (end of each sprint) | | | | | | | | |
| **Development** | | | | | | | | |
| Setup, architecture, database design | | | | | | | | |
| Authentication and user isolation | | | | | | | | |
| Upload, parsing, chunking | | | | | | | | |
| Embeddings, Qdrant indexing, retrieval | | | | | | | | |
| LLM answers, context and citations | | | | | | | | |
| Frontend: auth, upload, chat, citations | | | | | | | | |
| Conversation history (backend and UI) | | | | | | | | |
| Hybrid retrieval, reranking, query rewriting | | | | | | | | |
| Error handling and responsiveness | | | | | | | | |
| **Testing** | | | | | | | | |
| Unit and API tests (every sprint) | | | | | | | | |
| Frontend validation | | | | | | | | |
| Integration, isolation and regression tests | | | | | | | | |
| **Documentation** | | | | | | | | |
| Backlog, design notes, sprint review notes | | | | | | | | |
| README, UML diagrams, report, screenshots | | | | | | | | |
| Demo preparation | | | | | | | | |
| **Milestones** | | **M1** | | **M2** | | **M3** | | **M4** |

Legend: Scrum events Development Testing Documentation **M1** ingestion increment **M2** first cited answer via API **M3** usable web application **M4** submission ready. Weeks are relative to the project start (Week 1 = first week).

### 5.1 Milestones

| Milestone | When | Measurable criteria |
|---|---|---|
| M1 Ingestion increment | End of Sprint 1 (Week 2) | Authenticated user uploads PDF, TXT and MD files; page-tagged chunks are produced; invalid files rejected; tests pass |
| M2 First cited answer | End of Sprint 2 (Week 4) | A question sent to the API returns an answer from the user's documents with citations; documents can be listed and deleted |
| M3 Usable web application | End of Sprint 3 (Week 6) | All Must-have user stories work in the browser; history works if delivered |
| M4 Submission ready | End of Sprint 4 (Week 8) | All acceptance criteria checked; README, diagrams, report, screenshots and demo complete |

### 5.2 If a 14-working-day deadline applies

A conventional Scrum sprint is two calendar weeks, and this plan uses four of them. A 14-working-day deadline is less than one quarter of that time and gives only about 42 hours at 3 hours per day. In that case:

- Use four **short sprints of 3-4 working days** each, keeping the same goals in the same order (foundation, core RAG, frontend, quality and submission).
- Keep only Must-have items, simplified where possible (for example, minimal styling and basic history only if time remains).
- Drop all Should-have and Could-have items: hybrid retrieval, reranking, query rewriting, the history sidebar and deployment.
- Even then, the Must-have scope (82 SP) is large for about 42 hours; agree the reduced scope with the supervisor before starting.

### 5.3 Relationship to the COCOMO estimate

COCOMO is a model-based estimate of total effort from code size (13.0 person-months, about 1,977 hours, over 6.6 months). The Agile plan instead fixes time and capacity (4 sprints, about 120 hours) and adjusts scope by priority. Basic COCOMO does not produce story points, and story points are not converted into hours here. The large difference between the two figures is recorded as a schedule risk (Section 8); sprint metrics (Section 9) will show early whether the planned scope is achievable.

## 6. Agile Roles and Responsibilities

One student carries out all roles. These are **responsibilities**, not separate team members.

| Role | Responsibilities |
| --- | --- |
| Product Owner | Maintain and prioritise the product backlog; write user stories and acceptance criteria; decide what is dropped if time runs short; incorporate supervisor feedback. |
| Scrum Master (self-management) | Keep the sprint rhythm; run planning, review and retrospective; keep the daily log; remove blockers or escalate them to the supervisor. |
| Developer | Design and implement backend, frontend, ingestion, retrieval and LLM integration; keep the code working at the end of each sprint. |
| Tester | Write unit, API and integration tests; run manual checks against acceptance criteria; log and track defects. |
| Documentation owner | Maintain the README, UML diagrams, sprint notes, report and screenshots; prepare the demonstration. |
| Supervisor (stakeholder) | Not a team role: reviews increments when available and gives feedback. |

# 7. Testing Strategy

Testing happens inside every sprint as part of the Definition of Done, not in a separate phase at the end.

| Test type | What is checked | Sprints |
|---|---|---|
| Unit tests | Parsers, chunking, embedding wrapper, BM25 and RRF fusion, reranking order, context and citation building, password hashing | 1-4 |
| API tests | Auth, upload, documents, query and history endpoints: valid requests, validation errors, 401/403/404 responses | 1-4 |
| Integration tests | Upload to parsing, chunking, embedding and Qdrant; query to retrieval, LLM and citations, with real PostgreSQL and Qdrant | 2-4 |
| Frontend validation | Type checks and lint; unit tests for the API client and citation helpers; manual browser checks of each user story | 3-4 |
| RAG answer and citation checks | Fixed set of sample documents and questions with known source passages; check that answers use them and citations point to them | 2-4 |
| Authorization and user-isolation tests | A second user cannot list, query, read history of, or delete the first user's data | 1-4 |
| Regression tests | Full automated suite run before each sprint review and before submission | 1-4 |

## 7.1 Measurable acceptance criteria

| Area | Criterion |
|---|---|
| Upload | 100% of valid sample PDF, TXT and MD files are accepted; 100% of unsupported, empty and oversized test files are rejected with a message |
| Question answering | For each question in the sample set, the answer uses content from the expected document; for questions with no answer in the documents, the system states that it could not find the information |
| Citations | Every citation marker in an answer matches a returned source, and each source shows document, page and text |
| History | Saved conversations reopen with all messages in order; deleted conversations cannot be accessed |
| Security | All protected endpoints reject requests without a valid token; all user-isolation tests pass |
| Quality gate | All automated tests pass at each sprint review and before submission |

> Results will be recorded during the sprints and attached as test evidence. No results are reported in this plan.

# 8. Risk Management

| Risk | Likelihood | Impact | Mitigation | Contingency action |
|---|---|---|---|---|
| Poor retrieval quality (irrelevant chunks, missed answers) | Medium | High | Tune chunk size/top-k in Sprint 2; add hybrid retrieval and reranking in Sprint 4; fixed sample question set | Limit scope to well-structured text documents; document known limitations |
| LLM API unavailable, rate-limited or over budget | Medium | Medium | Small low-cost model; spending limit; fake LLM in automated tests | Show a clear error; demo with a recorded run; switch to a compatible provider |
| Local models slow or too heavy (embedding, reranker) | Medium | Medium | Small models, loaded once; processing off the event loop; file size limit | Drop reranking (Should-have); reduce candidate count |
| Integration problems between frontend, API, PostgreSQL and Qdrant | Medium | High | Thin end-to-end increment by end of Sprint 2; typed API client; integration tests | Use sprint buffer; postpone Should-have UI items |
| Schedule overrun (COCOMO suggests far more effort than planned capacity) | High | High | MoSCoW priorities; Must-haves first; track points per sprint | Move Should/Could items out; keep Must-have scope only |
| Data leak between users | Low | High | User ID filter on every query; isolation tests in every sprint from Sprint 1 | Block release until isolation tests pass |
| PDF extraction failures (scanned or complex PDFs) | High | Medium | Test several PDF types in Sprint 1; OCR declared out of scope | Reject files with no text and explain why |
| Deployment problems (memory limits, secrets, cost) | Medium | Low | Deployment is a Could-have; local demo is the default | Present locally with screenshots and recording |

Risks are reviewed at every sprint planning and retrospective. The main contingency mechanism is scope: Could-have, then Should-have items are removed before any Must-have item is affected.

# 9. Agile Metrics and Monitoring

| Metric | How it is measured | Used for |
|---|---|---|
| Planned vs completed backlog items | Story points and items marked Done (meeting the Definition of Done) per sprint | Measuring velocity from Sprint 1 onward and re-planning later sprints |
| Sprint progress | Task board and a simple burndown of remaining story points, updated each working day | Spotting slippage early within the sprint |
| Defect count | Open and closed defects logged per sprint, by severity | Deciding whether to spend capacity on fixes before new features |
| Acceptance criteria met | Number of acceptance criteria verified out of the total for the sprint's items | Sprint review and final acceptance |
| Retrospective findings | Improvement actions recorded and checked at the next retrospective | Continuous process improvement |
| Hours logged (optional) | Hours spent per sprint compared with the ~30-hour capacity | Comparing real effort with the COCOMO and capacity assumptions |

> No historical velocity exists, so none is assumed. The first real velocity value will be available at the end of Sprint 1.

# 10. Final Deliverables and Acceptance Criteria

| Deliverable | Acceptance criteria |
|---|---|
| Working RAG Assistant | Runs locally with the documented commands; all Must-have user stories pass their acceptance criteria |
| Frontend and backend source code | In a Git repository with a clear structure; no secrets committed |
| Document ingestion and retrieval pipeline | PDF, TXT and MD files are parsed, chunked, embedded and indexed; dense (and, if delivered, hybrid and reranked) retrieval works |
| Question answering with citations | Answers are grounded in retrieved content and every citation is traceable to a source |
| Database integration | PostgreSQL stores users, documents, conversations and messages; Qdrant stores vectors with user and document filters |
| Automated tests and test evidence | Test suite runs with one command; output and the end-to-end checklist are included |
| UML class and activity diagrams | Diagrams match the implemented design |
| README and setup instructions | A new machine can be set up by following them |
| Project report and screenshots | Covers login, upload, answer with citations, history and error handling |
| Deployment (if required and feasible) | Public URL works; otherwise a local demonstration and recording are provided |

- ☐ Working application demonstrated
- ☐ Source code submitted
- ☐ Tests and test evidence
- ☐ UML class and activity diagrams
- ☐ README and setup instructions
- ☐ Project report, COCOMO estimate and Agile plan
- ☐ Screenshots and demonstration
- ☐ Deployment link (only if required)

# 11. Conclusion

The Agile approach delivers the RAG Assistant in four two-week sprints, each ending with a tested, working increment: ingestion, a cited answer through the API, a usable web application, and finally a hardened, documented submission. Testing is built into every sprint through the Definition of Done, and the prioritised backlog keeps the Must-have core safe if time runs short. Because the COCOMO estimate indicates more effort than the student schedule allows, scope management and early velocity measurement are central to this plan. Actual progress will be recorded during the sprints; no outcome is assumed in advance.